

Project Topic:

Estimation of Obesity Levels Based On Eating Habits and Physical Condition

This dataset include data for the estimation of obesity levels in individuals from the countries of Mexico, Peru and Colombia, based on their eating habits and physical condition. The data contains 17 attributes and 2111 records, the records are labeled with the class variable NObesity (Obesity Level), that allows classification of the data using the values of Insufficient Weight, Normal Weight, Overweight Level I, Overweight Level II, Obesity Type I, Obesity Type II and Obesity Type III.

This project is aimed at predicting obesity levels in individuals from Mexico, Peru, and Colombia based on their eating habits and physical conditions by developing and deploying a Machine Learning model.

Link to data:

<https://archive.ics.uci.edu/dataset/544/estimation+of+obesity+levels+based+on+eating+habits>



```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsOneClassifier
from sklearn.metrics import accuracy_score
```

The attributes of the dataset are described below:

Variable Name	Type	Description
Gender	Categorical	
Age	Continuous	
Height	Continuous	
Weight	Continuous	
family_history_with_overweight	Binary	Has a family member suffered or suffers from overweight?
FAVC	Binary	Do you eat high caloric food frequently?

Variable Name	Type	Description
FCVC	Integer	Do you usually eat vegetables in your meals?
NCP	Continuous	How many main meals do you have daily?
CAEC	Categorical	Do you eat any food between meals?
SMOKE	Binary	Do you smoke?
CH2O	Continuous	How much water do you drink daily?
SCC	Binary	Do you monitor the calories you eat daily?
FAF	Continuous	How often do you have physical activity?
TUE	Integer	How much time do you use technological devices such as cell phone, videogames, television, computer and others?
CALC	Categorical	How often do you drink alcohol?
MTRANS	Categorical	Which transportation do you usually use?
NObeyesdad	Categorical	Obesity level

```
In [3]: #Load data
data=pd.read_csv(r'C:\Users\Owner\OneDrive\Desktop\My Projects\ObesityDataSet_raw_a
data.sample(5)
```

```
Out[3]:
```

	Gender	Age	Height	Weight	family_history_with_overweight	FAVC	FCV
691	Male	18.000000	1.786758	51.524444	no	yes	2.9154
17	Female	29.000000	1.530000	78.000000	no	yes	2.0000
876	Female	16.093234	1.608914	65.000000	yes	yes	1.3210
351	Female	26.000000	1.650000	60.000000	no	no	3.0000
205	Female	23.000000	1.600000	78.000000	yes	yes	2.0000

Eploratory Data Analysis (EDA)

```
In [280... data.isnull().sum()
```

```
Out[280... Gender      0
Age      0
Height   0
Weight   0
family_history_with_overweight  0
FAVC     0
FCVC     0
NCP      0
CAEC     0
SMOKE    0
CH20     0
SCC      0
FAF      0
TUE      0
CALC     0
MTRANS   0
NObeyesdad  0
dtype: int64
```

```
In [281... data.shape
```

```
Out[281... (2111, 17)
```

```
In [282... data.nunique()
```

```
Out[282... Gender      2
Age      1402
Height   1574
Weight   1525
family_history_with_overweight  2
FAVC     2
FCVC     810
NCP      635
CAEC     4
SMOKE    2
CH20     1268
SCC      2
FAF      1190
TUE      1129
CALC     4
MTRANS   5
NObeyesdad  7
dtype: int64
```

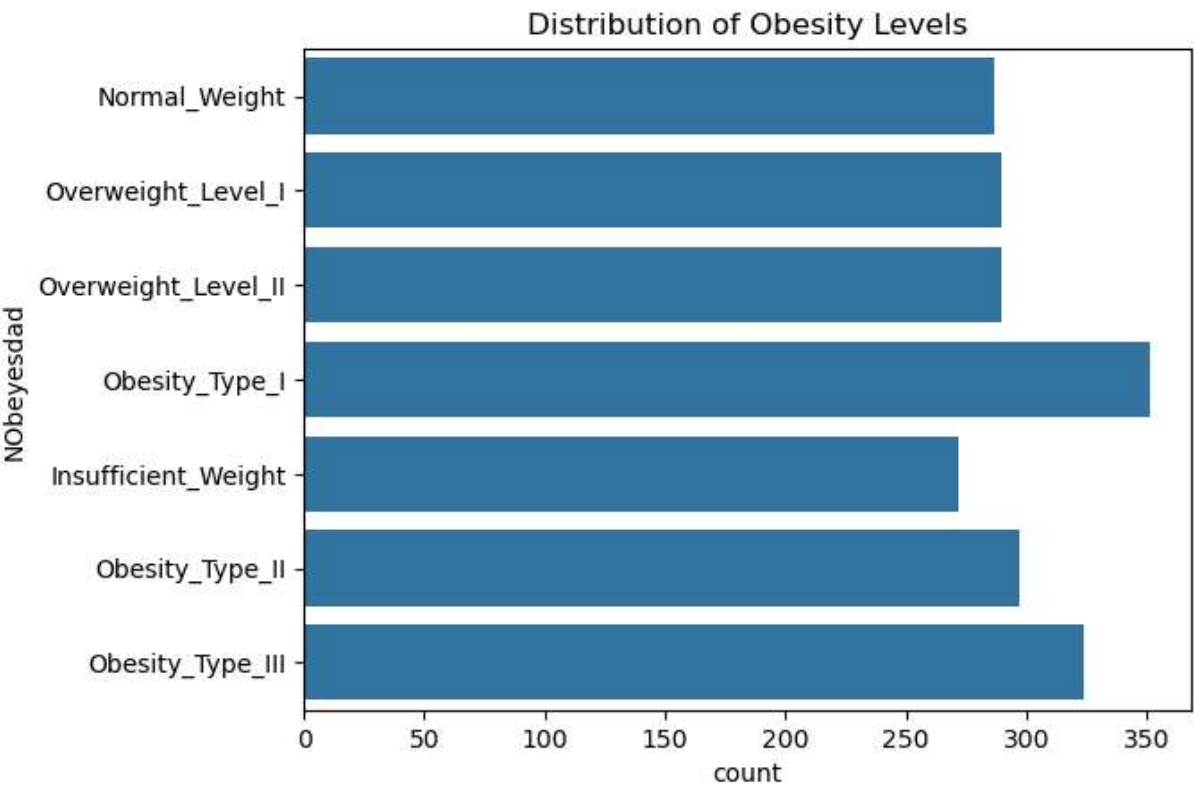
```
In [283... data.describe()
```

Out[283...

	Age	Height	Weight	FCVC	NCP	CH2O	
count	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111
mean	24.312600	1.701677	86.586058	2.419043	2.685628	2.008011	1
std	6.345968	0.093305	26.191172	0.533927	0.778039	0.612953	0
min	14.000000	1.450000	39.000000	1.000000	1.000000	1.000000	0
25%	19.947192	1.630000	65.473343	2.000000	2.658738	1.584812	0
50%	22.777890	1.700499	83.000000	2.385502	3.000000	2.000000	1
75%	26.000000	1.768464	107.430682	3.000000	3.000000	2.477420	1
max	61.000000	1.980000	173.000000	3.000000	4.000000	3.000000	3

Visualize the distribution of the target variable to understand the class balance

```
In [4]: # Distribution of target variable
%matplotlib inline
sns.countplot(y='NObeyesdad', data=data)
plt.title('Distribution of Obesity Levels')
plt.show()
```



This shows that the dataset is fairly balanced and does not require any special attention in terms of biased training.

Data Preprocessing

Feature scaling

```
In [5]: # Standardizing continuous numerical features
continuous_columns = data.select_dtypes(include=['float64']).columns.tolist()

scaler = StandardScaler()
scaled_features = scaler.fit_transform(data[continuous_columns])

# Converting to a DataFrame
scaled_df = pd.DataFrame(scaled_features, columns=scaler.get_feature_names_out(cont

# Combining with the original dataset
scaled_data = pd.concat([data.drop(columns=continuous_columns), scaled_df], axis=1)
```

```
In [305... scaled_data.head()
```

```
Out[305...      Gender  family_history_with_overweight  FAVC      CAEC  SMOKE  SCC      CALC
0  Female                                yes    no  Sometimes    no    no          no  Pu
1  Female                                yes    no  Sometimes    yes   yes  Sometimes  Pu
2   Male                                yes    no  Sometimes    no    no  Frequently  Pu
3   Male                                no     no  Sometimes    no    no  Frequently
4   Male                                no     no  Sometimes    no    no  Sometimes  Pu
```



One-hot encoding

```
In [6]: # Identifying categorical columns
categorical_columns = scaled_data.select_dtypes(include=['object']).columns.tolist()
categorical_columns.remove('NObeyesdad') # Exclude target column

# Applying one-hot encoding
encoder = OneHotEncoder(sparse_output=False, drop='first')
encoded_features = encoder.fit_transform(scaled_data[categorical_columns])

# Converting to a DataFrame
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(c

# Combining with the original dataset
prepped_data = pd.concat([scaled_data.drop(columns=categorical_columns), encoded_df
```

```
In [7]: prepped_data.sample(6)
```

Out[7]:

	NObeyesdad	Age	Height	Weight	FCVC	NCP	CH2O
631	Insufficient_Weight	-0.794397	0.128562	-1.397216	-2.647930	1.689740	-1.644905
1068	Overweight_Level_II	-0.410348	-0.464013	-0.251521	-0.785019	0.404153	-0.013073
1308	Obesity_Type_I	-0.206889	0.364287	0.261339	-0.785019	0.404153	-1.238078
1516	Obesity_Type_II	1.323209	-0.067862	0.619617	0.704464	-0.714426	-1.644905
904	Overweight_Level_I	-0.638458	1.134837	-0.048499	-0.785019	0.404153	-0.065274
495	Normal_Weight	-0.837360	1.054029	-1.015318	1.088342	-2.167023	-1.644905

6 rows × 24 columns



Encoding the target variable

In [8]: `# Encoding the target variable`

```
prepped_data['NObeyesdad'] = prepped_data['NObeyesdad'].astype('category').cat.codes
prepped_data.head(10)
```

Out[8]:

	NObeyesdad	Age	Height	Weight	FCVC	NCP	CH2O	FAF
0	1	-0.522124	-0.875589	-0.862558	-0.785019	0.404153	-0.013073	-1.188039
1	1	-0.522124	-1.947599	-1.168077	1.088342	0.404153	1.618759	2.339750
2	1	-0.206889	1.054029	-0.366090	-0.785019	0.404153	-0.013073	1.163820
3	5	0.423582	1.054029	0.015808	1.088342	0.404153	-0.013073	1.163820
4	6	-0.364507	0.839627	0.122740	-0.785019	-2.167023	-0.013073	-1.188039
5	1	0.738817	-0.875589	-1.282647	-0.785019	0.404153	-0.013073	-1.188039
6	1	-0.206889	-2.162001	-1.206267	1.088342	0.404153	-0.013073	-0.012109
7	1	-0.364507	-0.661187	-1.282647	-0.785019	0.404153	-0.013073	2.339750
8	1	-0.049271	0.839627	-0.862558	1.088342	0.404153	-0.013073	-0.012109
9	1	-0.364507	0.196421	-0.709799	-0.785019	0.404153	-0.013073	-0.012109

10 rows × 24 columns



In [9]: `#Confirm encoded target classes`

```
prepped_data['NObeyesdad'].unique()
```

Out[9]: `array([1, 5, 6, 2, 0, 3, 4], dtype=int8)`

To evaluate the correlation (relationship) of the target variable with the input features, it will be convenient to create a correlation matrix of the input variables with the target variable.

```
In [10]: #compute the pairwise correlation matrix
corr_matrix=prepped_data.corr()
corr_matrix.head()
```

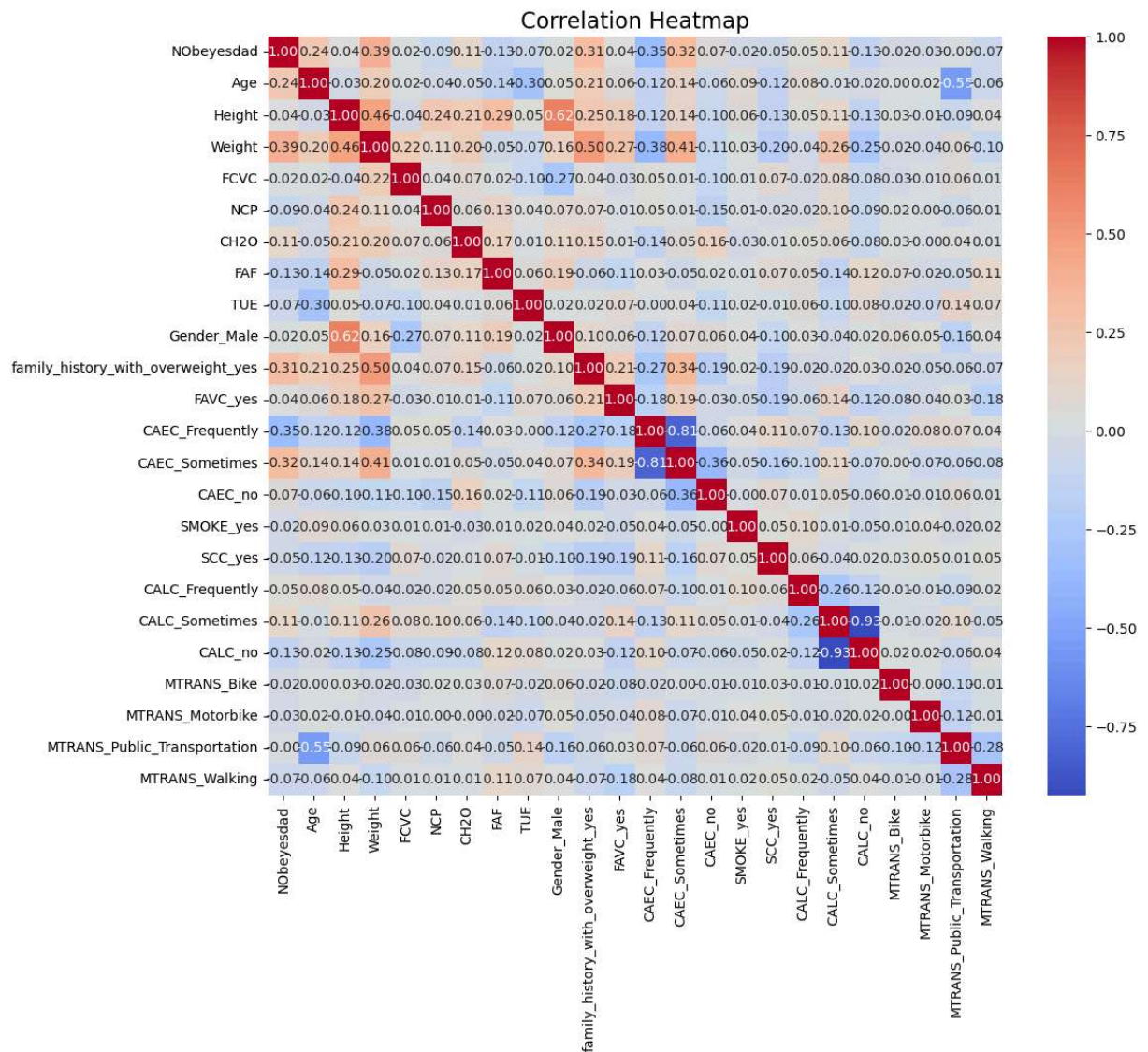
```
Out[10]:
```

	NObeyesdad	Age	Height	Weight	FCVC	NCP	CH2O
NObeyesdad	1.000000	0.236170	0.038986	0.387643	0.018522	-0.092616	0.108868
Age	0.236170	1.000000	-0.025958	0.202560	0.016291	-0.043944	-0.045304
Height	0.038986	-0.025958	1.000000	0.463136	-0.038121	0.243672	0.213376
Weight	0.387643	0.202560	0.463136	1.000000	0.216125	0.107469	0.200575
FCVC	0.018522	0.016291	-0.038121	0.216125	1.000000	0.042216	0.068461

5 rows × 24 columns



```
In [12]: #Produce the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap", fontsize=16)
plt.show()
```

The heatmap is a pictorial representation of the correlation matrix. The diagonal entries are 1 because the correlation of a variable to itself is 1. Our main interest here, is to investigate the relationship between each the independent variable(attribute) and the dependent(target)variable. Looking at the first column, we see these input variables (**Age-0.24, Weight-0.39, family_history_with_overweight_yes-0.31, CAEC_Frequently-0.35, CAEC_Sometimes-0.32**) show somewhat significant (moderate positive) correlations with the target variable. I'm very careful here because zero correlation does not mean there isn't a relationship, but the relationship might be non-linear. Hence, we keep the data for our model building.

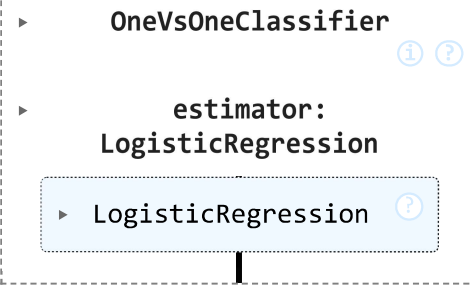
Split data into dependent(target) and independent(explanatory) variables.

```
In [13]: X=prepped_data.drop('NObyesdad', axis=1)
y=prepped_data['NObyesdad']
```

Model training and evaluation


```
In [17]: # Training logistic regression model using One-vs-One
X_train,X_test,y_train, y_test=train_test_split(X,y, test_size=0.2, random_state=42)
model_ovo = OneVsOneClassifier(LogisticRegression(max_iter=1000))
model_ovo.fit(X_train, y_train)
```

```
Out[17]:
```



```
In [18]: # Predictions
y_pred_ovo = model_ovo.predict(X_test)

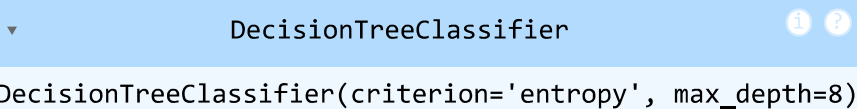
# Evaluation metrics for OvO
print("One-vs-One (OvO) Strategy")
print(f"Accuracy: {np.round(100*accuracy_score(y_test, y_pred_ovo),2)}%")
```

One-vs-One (OvO) Strategy
Accuracy: 93.38%

```
In [19]: from sklearn.tree import DecisionTreeClassifier
```

```
In [20]: model = DecisionTreeClassifier(criterion="entropy", max_depth = 8)
model.fit(X_train, y_train)
```

```
Out[20]:
```



```
In [21]: y_pred=model.predict(X_test)
```

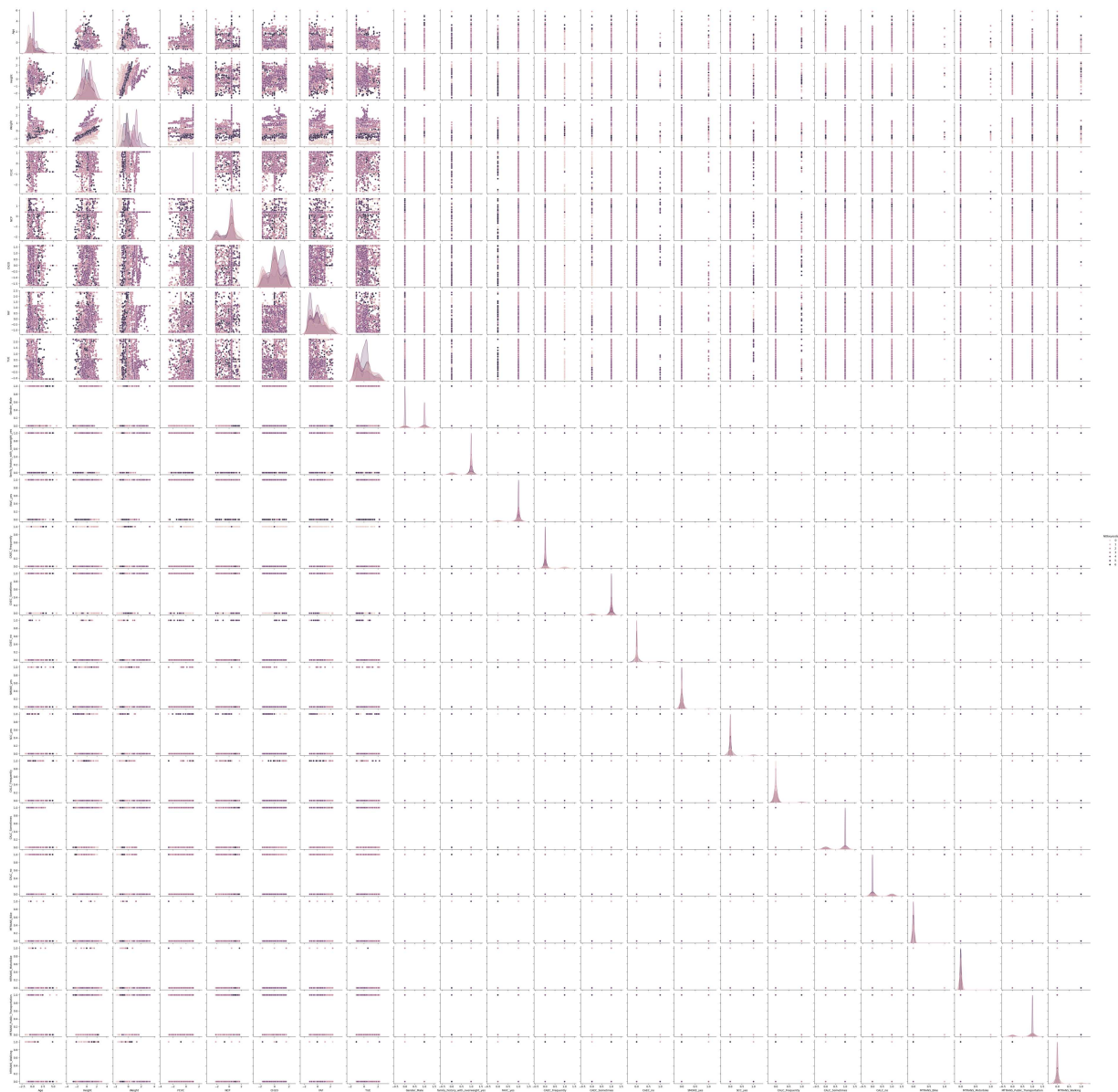
```
In [52]: #Evaluate model accuracy
print('Decision Tree method:')
print(f"model accuracy is: {round(model.score(X_test,y_test)*100,2)}%")
```

Decision Tree method:
model accuracy is: 94.33%

The two models seem to perform closely well. I decided to use the Decision tree because the algorithm captures non-linear relationships that may exist in the data. Also, the Decision tree can handle interactions automatically.

```
In [53]: %matplotlib inline
plt.figure(figsize=(6, 3))
sns.pairplot(prepped_data, hue='NObeyesdad')
plt.show()
```

<Figure size 600x300 with 0 Axes>



In []: